

BI Woche 3 Airport

Aufgabenblatt (BI): R + MySQL (Flughafen-Datenbank) – Dashboard-Query in 5 Aufgaben

Werkzeuge (erlaubt): Nur R, RStudio, MySQL (z.B. MySQL Workbench/CLI)

Abgabe: Ein R-Skript (`dashboard_airport.R`) + die finale SQL-Abfrage (im Skript) + 2–3 Grafiken/Tabellen in R

Ziel: Ihr baut in 5 Aufgaben eine Datenpipeline, deren Kern eine **komplexe Abfrage** ist, die ein **Dashboard** für typische Flughafen-Operationen speist.

Kontext: “Operations-Dashboard” für einen Flughafen

Ein Flughafen-Operations-Team will täglich sehen:

- Wie viele Flüge starten/landen heute (oder in einem Datumsbereich)?
- Welche Airlines / Ziele verursachen die meisten Verspätungen?
- Welche Routen sind am stärksten ausgelastet (Ticket-/Passagieraufkommen)?
- Welche Zeitfenster sind Peak Hours?
- Welche Flüge sind “kritisch” (z.B. starke Verspätung, hohe Auslastung, hohe Erlöse)?

Ihr erstellt dafür einen **Dashboard-Datensatz** aus MySQL und analysiert ihn in R.

Annahmen zur Datenbank (ihr müsst die echten Tabellen prüfen!)

Eure DB kann andere Namen haben. Nutzt zuerst:

- `SHOW TABLES;`
- `DESCRIBE <tabelle>;`
- `SELECT * FROM <tabelle> LIMIT 10;`

Typische Tabellen (Beispiel):

- `airlines(airline_id, name, country)`
- `airports(airport_id, iata_code, name, city, country)`
- `flights(flight_id, airline_id, origin_id, destination_id, sched_departure, actual_departure, sched_arrival, actual_arrival, status)`
- `tickets(ticket_id, flight_id, price, booking_date)`
- **Optional:** `passengers(passenger_id, ticket_id, ...)` oder `bookings(...)`

Falls eure DB keine “delay”-Felder hat: Ihr berechnet Verspätung aus `actual_departure - sched_departure` (in Minuten), sofern Zeiten vorhanden sind.

Aufgabe 1 (R): Verbindung herstellen + Struktur erkunden (Pflicht)

Ziel: Ihr könnt aus R auf MySQL zugreifen und Tabellen/Spalten verstehen.

To-Do:

1. Erstellt in R ein Verbindungsobjekt `con`.
2. Listet alle Tabellen auf.
3. Wählt **mindestens 3 Tabellen** aus (z.B. `flights`, `airports`, `airlines`, `tickets`) und zeigt die ersten 10 Zeilen.

Benötigte R-Befehle (Hinweise):

- `library(DBI)`
- `dbConnect(...)`
- `dbListTables(con)`
- `dbGetQuery(con, "SQL...")`
- `dbDisconnect(con)`

Output (im Skript als Kommentar dokumentieren):

- Welche Tabellen sind relevant fürs Dashboard?
- Welche Felder sind Schlüssel (IDs) und welche sind Maße (Preis, Zeiten, Status)?

Aufgabe 2 (MySQL + R): “Clean KPI” – Verspätungen & Status normalisieren

Ziel: Ihr erzeugt aus `flights` eine saubere Basis für Analysen.

To-Do:

1. Erstellt eine Abfrage, die **pro Flug** folgende Felder liefert:
 - Flug-ID
 - Airline (Name)
 - Origin IATA, Destination IATA
 - Datum (aus Abflugzeit)
 - geplante Abflugzeit, tatsächliche Abflugzeit (falls vorhanden)
 - **departure_delay_min** (in Minuten; negative Werte auf 0 setzen)
 - **Status-Klasse**: z.B. `ON_TIME`, `DELAYED`, `CANCELLED`
2. Zieht das Ergebnis nach R und prüft:
 - Gibt es `NA/NULL`? Wie viele?
 - Durchschnittliche Delay-Minuten

SQL-Hinweise:

- `TIMESTAMPDIFF(MINUTE, sched_departure, actual_departure)`
- `CASE WHEN ... THEN ... END`
- Negative Delays auf 0: `GREATEST(delay, 0)` (oder `CASE`)

R-Hinweise:

- `summary(df)`

- `mean(df$departure_delay_min, na.rm=TRUE)`

Aufgabe 3 (R): Parameterisierung

Ziel: Das Dashboard soll für **einen Flughafen** (z.B. ZRH) und einen **Datumsbereich** laufen.

To-Do:

1. Legt in R Parameter an:
 - `airport_iata <- "ZRH"` (oder euer Flughafen)
 - `date_from <- "2026-01-01"`
 - `date_to <- "2026-01-31"`
2. Baut daraus **eine SQL-Abfrage als String**, die Flüge filtert:
 - entweder **Abflug von** `airport` (origin) **oder Ankunft in** `airport` (destination)
 - im Datumsbereich

R-Hinweis (Stringbau):

- `sprintf(...)` oder `paste0(...)`

Achtung: Keine SQL-Injection in echten Systemen – hier genügt `sprintf`, aber sauber bleiben.

Aufgabe 4 (MySQL): Dashboard-KPIs als aggregierte “Kachelwerte”

Ziel: Ihr erzeugt Kennzahlen, wie sie auf einem Dashboard als “Kacheln” stehen.

To-Do (SQL):

Schreibt eine Abfrage, die für den Datumsbereich und den Flughafen liefert:

- `total_flights`
- `cancelled_flights`
- `pct_cancelled`
- `avg_departure_delay_min`
- `p90_departure_delay_min` (*falls möglich; wenn nicht, begründet Alternative*)
- `total_ticket_revenue` (*aus tickets; wenn Tickets fehlen: Alternative Kennzahl wählen*)

Hinweise:

- Umsatz: `Join tickets -> flights`
- Prozent: `100.0 * cancelled / total`
- p90: Wenn MySQL 8 + Window Functions verfügbar: ihr dürft `PERCENT_RANK()` / `NTILE()` / `PERCENTILE_CONT()` nutzen (je nach Support).
Wenn nicht: nutzt eine dokumentierte Näherung (z.B. sortieren + indexbasierter Wert in R).

In R:

- Druckt die KPI-Tabelle (`print(kpis)`), speichert sie optional als CSV.

Aufgabe 5 (Kernaufgabe): Eine komplizierte Abfrage für ein Operations-Dashboard

Dashboard-Frage (typische Flughafen-Aufgabe)

“Welche Routen und Airlines verursachen im gewählten Zeitraum am Flughafen die größten operativen Risiken – gemessen an Verspätung UND Volumen (Anzahl Flüge) UND Erlös?”

Anforderungen an die finale Abfrage

Schreibt **eine** SQL-Abfrage, die einen Dashboard-Datensatz liefert – **pro Tag und Stunde** (Time Bucket) sowie pro Airline und Route.

Granularität (Gruppierung):

- `date` (YYYY-MM-DD)
- `hour` (0–23) aus geplanter Abflugzeit (oder tatsächlicher, begründen!)
- `airline_name`
- `origin_iata`
- `destination_iata`

Metriken (Spalten im Ergebnis):

1. `flights_count`
2. `cancelled_count`
3. `avg_delay_min` (Departure delay; negative auf 0)
4. `delayed_15m_count` (Anzahl Flüge mit `delay >= 15`)
5. `delay_rate_15m` = `delayed_15m_count / flights_count`
6. `ticket_revenue` (SUM der Ticketpreise für diese Flights im Bucket)
7. **risk_score** (euer definierter Score, z.B.):
$$\text{risk_score} = (\text{avg_delay_min} * \text{delay_rate_15m}) * \text{LOG}(1 + \text{flights_count}) + (\text{ticket_revenue} / 1000)$$

(Ihr dürft eine andere sinnvolle Formel nehmen – aber dokumentieren!)

Filter:

- Nur Flüge, bei denen `origin_iata = airport_iata` OR `destination_iata = airport_iata`
- Nur Datumsbereich `date_from` bis `date_to`

Sortierung:

- Ausgabe soll standardmäßig nach `risk_score` DESC sortieren.

“Kompliziert”-Kriterien (mindestens 5 davon müssen vorkommen)

Eure Query gilt als “kompliziert”, wenn sie **mindestens 5** der folgenden Features nutzt:

- Mindestens **4 Tabellen** (z.B. `flights`, `airports` (2x), `airlines`, `tickets`)

- Mindestens **2 JOINS** auf Dimensionstabellen
- Mindestens **2 CASE-WHEN**-Berechnungen
- Berechnung von **Time Buckets** (DATE + HOUR)
- Mindestens **3 Aggregationen** (COUNT, SUM, AVG, etc.)
- Erzeugung eines **Rates** (Quotient) mit NULL-sicherer Division
- Berechnung eines **Scores** aus mehreren Metriken
- Optional (Bonus): CTE (WITH ...) oder Subquery zur Voraggregation Tickets → Flights

Abgabe in R (Pflicht)

1. Baut die Query in R (mit euren Parametern).
2. Ladet sie nach R als `dashboard_df`.
3. Erzeugt mindestens **2 Visualisierungen** in R:
 - (A) **Top-10 risk routes** (z.B. Balkendiagramm)
 - (B) **Delay trend** über Zeit (Linie nach Datum; ggf. facettenartig via Schleife je Airline)

Ohne Zusatzpakete ist ok (base R). ggplot2 ist erlaubt, falls installiert.

R-Hinweise (minimal):

- `order(...), head(...)`
- `aggregate(...)`
- `plot(...), barplot(...)`

Qualitätsanforderungen

- Eure Query muss **reproduzierbar** laufen (ohne manuelles Nachklicken).
- Keine "SELECT *" in der finalen Query.
- Jede abgeleitete Spalte sauber benennen (Aliases).
- Annahmen dokumentieren (z.B. wenn Status-Feld anders heißt).

Bonus (optional, +)

- Exportiert den Dashboard-Datensatz als `dashboard_dataset.csv`
- Baut eine kleine "Alert-Liste": Flüge mit `delay >= 60` ODER `cancelled` UND `revenue` im oberen Quartil.

Vorlage: R-Skript-Struktur (ohne Lösung)

```
library(DBI)
library(RMySQL)

# 1) Parameter
airport_iata <- "FRA"
date_from <- "2026-01-01"
date_to <- "2026-01-31"

# 2) Connect
con <- dbConnect(
  MySQL(),
```

```

    host = "localhost",
    user = "root",
    password = "XXX",
    dbname = "airport_db",
    port = 3306
)

# 3) Explore
# dbListTables(con)
# dbGetQuery(con, "SHOW TABLES;")

# 4) FINAL DASHBOARD QUERY (Aufgabe 5)
# sql <- sprintf(" ... ", airport_iata, date_from, date_to)
# dashboard_df <- dbGetQuery(con, sql)

# 5) Analyse & Plots
# head(dashboard_df)
# ...

# 6) Disconnect
dbDisconnect(con)

```

Ziel der Übung ist:

1. dashboard_airport.R (kommentiert, lauffähig)
2. SQL-Abfrage (in R)
3. 3 Outputs (Tabellen/Plots) in R, die zeigen, dass der Dashboard-Datensatz funktioniert